



LESSONS LEARNED
LITPATH AI: SMART PATHFINDER FOR THESES AND DISSERTATIONS

**ASIA PACIFIC COLLEGE
3 HUMABON, MAGALLANES
MAKATI CITY, METRO MANILA 1232**

MAY 2026





TABLE OF CONTENTS

INTRODUCTION	3
LESSONS LEARNED APPROACH	3
LESSONS LEARNED FROM THIS PROJECT	4
LESSONS LEARNED KNOWLEDGE BASE / DATABASE.....	8
LESSONS LEARNED APPLIED FROM PREVIOUS PROJECTS	9
PROCESS IMPROVEMENT RECOMMENDATIONS	10





INTRODUCTION

Capturing lessons learned is an integral part of every project and serves several purposes. While the finalization of a formal lessons learned document is completed during the project closeout process, capturing lessons learned should occur throughout the project lifecycle to ensure all information is documented in a timely and accurate manner. The lessons learned document serves as a valuable tool for use by other project managers within an organization who are assigned similar projects. This document should not only describe what went wrong during a project and suggestions to avoid similar occurrences in the future, but it should also describe what went well and how similar projects may benefit from this information. This document should be communicated to the project sponsor and Project Management Office (PMO) for inclusion in the organizational assets and archives as part of the lessons learned database. If the organization does not have a PMO then other, formal means of communicating the lessons learned should be utilized to ensure all project managers are included.

Capturing lessons learned is an integral part of every project and serves several important purposes. While the finalization of a formal lessons learned document is completed during the project closeout process, capturing lessons learned should occur throughout the project lifecycle to ensure all information is documented in a timely and accurate manner. The lessons learned document serves as a valuable tool for future project managers assigned to similar projects, describing not only what went wrong and how to avoid similar occurrences, but also what went well and how future projects may benefit from these practices.

The purpose of this Lessons Learned document for the LitPath AI: Smart Pathfinder for Theses and Dissertations project is to capture the project's lessons in a formal document for use by other project managers on similar future academic technology projects. This document may be used during new project planning to understand what problems occurred, how they were handled, and how they may be avoided. It also details what went well and why, so that other project teams may apply these practices. This document will be formally communicated to the Project Sponsor and stored in the organizational assets and archives via OpenProject and SharePoint.

LESSONS LEARNED APPROACH

The lessons learned approach describes how the document will be created, what it will consist of, and how lessons will be categorized. It is important that the lessons learned approach is covered in the initial stages of project planning. The reason for this is that a methodology along with an appropriate set of tools should be established to capture these lessons throughout the project's lifecycle. A project journal is one example of a tool to capture these lessons. If no thought is given to lessons learned until project closeout then it is likely that many lessons and



details will be omitted from the document. The contents of the lessons learned document should also be determined ahead of time. They should be detailed enough to provide value for future use and the contents should be consistent with other lessons learned documents or organizational standards. The categorization of lessons learned is another consideration. Many organizations categorize lessons by project lifecycle phase or by the knowledge area that the lesson applies to.

The lessons learned approach describes how this document was created, what it consists of, and how lessons are categorized. For the LitPath AI project, lessons learned were captured continuously throughout the project lifecycle rather than only at close out, ensuring that details and context were preserved accurately as events occurred.

Lessons were gathered through the following methods:

1. **Sprint Retrospectives in OpenProject:** At the end of each two-week sprint, the HecTech team conducted retrospectives to reflect on what went well, what did not, and what could be improved. These retrospective notes served as the primary source of lessons learned throughout the project.
2. **Team Meetings and Daily Standups:** Issues, blockers, and successes raised during daily standups and weekly team meetings were logged and tracked, providing real-time insights into team dynamics, technical challenges, and process effectiveness.
3. **Group Journal and Log Entries:** The team maintained a group log documenting significant events, decisions, and observations throughout the project, serving as a chronological reference for compiling lessons learned during the closeout phase.

The lessons learned in this document are categorized by project knowledge area, consistent with standard project management practice. Knowledge areas represented include cost management, risk management, time management, scope management, human resource management, integration management, and technology and infrastructure management. Knowledge areas for which no lessons were documented are omitted.

LESSONS LEARNED FROM THIS PROJECT

The lessons learned must be communicated in a consistent manner. In addition to the categorization and description of the lesson, it is important to state what the impact was and provide a recommendation for project managers to consider on future projects.



The following table presents the lessons learned for the LitPath AI project, categorized by project knowledge area. Each lesson includes a description of the issue or success, its impact on the project, and a recommendation for consideration on similar future projects. Both challenges and successes are documented to provide a balanced and complete reference.

Category	Issue Name	Problem/Success	Impact	Recommendation
Cost Management	Zero-Cost Development Using Open-Source Tools	SUCCESS — The entire development phase was completed using open-source and free-tier tools (React.js, Django, SentenceTransformers, ChromaDB, Supabase) with no licensing or infrastructure costs.	The project was delivered at zero direct cost to both HecTech and DOST-STII, demonstrating that production-quality AI systems can be built in an academic setting without financial expenditure.	Thoroughly evaluate open-source and free-tier alternatives before committing to paid tools. A technology cost assessment at the planning stage can prevent unnecessary expenditure.
Risk Management	Early Identification of Gemini AI API Dependency	SUCCESS — The team identified early that the system's AI features depended entirely on the Gemini AI API. A ChromaDB-based semantic search fallback was planned and implemented before deployment.	Impact was minimal because fallback measures were in place before integration testing began, preventing critical failures during user testing and demonstrations.	Always assess third-party API dependencies at the start of development. Plan and implement fallback mechanisms before integration testing, not after. Include API rate limits and downtime scenarios in the risk register.



Time Management	Agile/Scrum Sprint Structure Kept Project on Track	SUCCESS — The team adopted a two-week sprint cadence with planning, daily standups, reviews, and retrospectives managed through OpenProject, providing consistent visibility into progress.	All major development milestones were completed on or close to schedule. The OpenProject sprint board served as a single source of truth, reducing miscommunication and keeping the team aligned.	Agile/Scrum methodology with short sprint cycles is strongly recommended for academic software projects. A dedicated project management tool such as OpenProject significantly improves team coordination and accountability.
	Codebase Restructuring Caused Mid-Project Delays	PROBLEM — During March to April 2026, performance optimization requirements led to significant codebase restructuring that was not fully anticipated in the original sprint plan.	The restructuring extended the development phase and caused it to run concurrently with Proxmox deployment and internal testing, stretching team capacity across multiple high-effort tasks simultaneously.	Address technical debt progressively throughout development rather than deferring it to a single optimization phase. Include a dedicated refactoring sprint in the project plan if a major restructuring is anticipated.



Scope Management	Admin Dashboard Required Additional Client Refinement Cycles	PROBLEM — The administrative analytics dashboard required more refinement cycles with DOST-STII than initially planned, with client feedback surfacing additional requirements after development had begun.	The admin dashboard remained in progress beyond its originally targeted completion point, requiring additional sprint effort to incorporate client-requested changes.	Client-facing features should undergo detailed requirements elicitation before full development begins. Prototyping or wireframing admin features early and presenting them to the client prevents late-stage scope changes.
Human Resource Management	Clear Role Distribution Prevented Communication Bottlenecks	SUCCESS — The HecTech team maintained clearly defined roles across the project, with each member taking ownership of specific technical areas while contributing collectively to documentation.	Role clarity prevented task overlap and confusion, reduced internal coordination time, and allowed the team to work in parallel across multiple features during the same sprint.	Clearly define and document team roles at the start of the project, even for small teams. Role clarity becomes especially important during high-intensity development phases when multiple features are built simultaneously.



Integration Management	Regular Adviser Consultations Kept Project Aligned	SUCCESS — The team maintained regular consultations with their academic adviser and co-adviser throughout the lifecycle, receiving timely technical and project management guidance.	Issues were identified and resolved early as a result of adviser feedback, preventing misalignment between deliverables and requirements and informing key technology and deployment decisions.	Establish a regular consultation cadence with advisers or client representatives from the project start. Regular check-ins provide higher value than limiting engagement to formal milestone reviews.
Technology & Infrastructure Management	Proxmox Deployment Overlapped with Active Development	PROBLEM — The Proxmox deployment and internal testing phase in April 2026 overlapped with ongoing development work. The team had not completed all development tasks before deployment activities began.	Team members managed both active code changes and deployment configuration simultaneously, increasing the risk of environment inconsistencies and requiring additional effort to stabilize the testing environment.	Establish a feature freeze milestone before beginning server deployment and internal testing. All active development should be completed before the deployment environment is configured.

LESSONS LEARNED KNOWLEDGE BASE / DATABASE

The Lesson Learned Knowledge Base contains historical information from previous projects. It is part of the organizational project assets and provides a valuable source of information to be used by similar projects in the future. All project lessons learned and other historical information need to be transferred to this knowledge/database in order to provide one centralized repository for ease of use. This should also include information on issues and risks as well as techniques that worked well which can be applied to future projects. Most lessons learned knowledge/databases contain large amounts of information, so it is important that there is a system for cataloging this information.

The Lessons Learned Knowledge Base contains historical information from completed projects and serves as a centralized repository for organizational project assets. All project lessons



learned, risk outcomes, issue logs, and techniques that worked well should be stored here to ensure ease of access and reference for future project teams.

The lessons learned for the LitPath AI project are stored across two platforms that serve as the team's organizational knowledge repository:

1. OpenProject: Sprint retrospective notes, issue logs, and change request records generated throughout the project lifecycle are archived within the LitPath AI project space in OpenProject, cataloged by sprint and knowledge area for structured retrieval.
2. SharePoint: The formal Lessons Learned document, along with all other project management documents — including the Business Case, Feasibility Study, SRS, System Design Document, Change Management Plan, Post-Project Review, and Project Acceptance — are archived in the team's SharePoint repository under the LitPath AI project folder, accessible to DOST-STII, APC faculty, and future project teams.

This information will be valuable for any future team assigned to develop or extend an AI-powered academic discovery system, particularly within a DOST or APC institutional context.

LESSONS LEARNED APPLIED FROM PREVIOUS PROJECTS

The lessons learned document might also state which historical lessons learned were used on this project. This information not only shows the value of the documentation of such lessons, but it also shows which lessons are consistently applied by other similar projects. It is important to reference not only what the lesson was but from which project it was associated with.

The LitPath AI project is the first project of its kind developed by the HecTech team. As such, no direct organizational lessons learned database from prior similar projects was available at the outset. However, the team drew on general best practices and publicly available knowledge from comparable academic and open-source AI development projects to inform key planning decisions:

1. Agile/Scrum methodology practices from APC's project management curriculum were applied in structuring the sprint cadence, retrospective process, and OpenProject setup, informing the team's approach to managing scope, tracking progress, and handling change requests.
2. Published guidance on RAG pipeline architecture and vector database integration informed the team's technology selection decisions, particularly the



- choice of ChromaDB and SentenceTransformers. Referencing community documentation and case studies helped the team avoid known pitfalls in semantic search implementation.
3. General software engineering best practices around API dependency risk — specifically the recommendation to plan fallback mechanisms for third-party services — were applied when integrating the Gemini AI API, leading to the early implementation of the ChromaDB-based fallback search.

PROCESS IMPROVEMENT RECOMMENDATIONS

It is important that once lessons learned are collected and documented that the organization approves and implement any process improvements identified. It is important for organizations to strive for continuous improvement and this portion of the lessons learned process is an integral step.

Based on the lessons learned documented in this report, the following process improvement recommendations are proposed for the HecTech team and for future project teams undertaking similar academic software development projects. It is important that these recommendations are reviewed, approved, and incorporated into future project planning processes to drive continuous improvement.

1. Introduce a Feature Freeze Milestone Before Deployment: Future projects should include a formal feature freeze checkpoint in the schedule, after which no new development is merged into the main codebase. This ensures the deployment and testing environment remains stable and that the team's attention is fully directed toward quality assurance and deployment preparation.
2. Conduct Client Requirements Elicitation for Admin Features Early: Administrative and client-facing features should be prototyped and reviewed with the client before full development begins. Wireframes or mockups presented during planning can surface client expectations early and prevent late-stage scope changes.
3. Address Technical Debt Within Each Sprint: Rather than deferring performance optimization to a dedicated phase, future teams should allocate approximately 20% of each sprint's capacity to refactoring and technical debt reduction. This prevents the accumulation of issues that require disruptive mid-project restructuring.
4. Formalize a Change Control Process from Day One: The Change Management Plan should be introduced and communicated to all stakeholders at the project kickoff. All scope or technical change requests should flow through the defined



CCB process from the first sprint, ensuring changes are evaluated and tracked consistently throughout the project lifecycle.

